Q3. Apply the CAP theorem to analyze a network partition scenario in a distributed blockchain system and illustrate the resulting trade-offs. (10 Marks) -Module 3

Introduction

The CAP theorem is a fundamental concept in distributed systems that states that a distributed system can provide only two out of the following three guarantees simultaneously when a network partition occurs:

1. Consistency (C): Every node in the distributed system sees the same and most recent data. All users receive consistent information.
2. Availability (A): Every request made to the system receives a response, even if some nodes are unavailable.
3. Partition Tolerance (P): The system continues operating despite communication failures between nodes due to network interruptions.

Network Partition Scenario in Blockchain

Consider a blockchain network consisting of five nodes distributed across different geographical locations. Due to a network failure, the network gets divided into two groups:

- Group A: Nodes 1, 2 and 3.
- Group B: Nodes 4 and 5.

Both groups cannot communicate with each other.

Suppose a user initiates a transaction to transfer 5 cryptocurrency coins from Account X to Account Y. Due to the network partition, both groups may receive different transaction requests.

Applying CAP Theorem

During the network partition, the blockchain must choose between consistency and availability while maintaining partition tolerance.

1. Consistency + Partition Tolerance (CP)

- The blockchain prioritizes maintaining a single, consistent ledger.
- Only the group capable of achieving the required consensus can validate transactions.
- The other group temporarily stops accepting transactions.
- This prevents conflicting transactions and double spending.

- However, some users may not be able to complete transactions during the partition.

Example: Hyperledger Fabric using a consensus configuration that requires a functioning quorum.

2. Availability + Partition Tolerance (AP)

- Both groups continue accepting transaction requests independently.
- Each group maintains its own version of the ledger.
- Users can continue performing transactions even without communication between groups.
- However, conflicting transactions or different ledger states may occur.
- Once connectivity is restored, reconciliation or conflict resolution may be required.

Example: A distributed blockchain design that allows independent transaction processing during network failures.

3. Consistency + Availability (CA)

- All nodes maintain consistent data.
- Every request receives a response.
- However, the system cannot guarantee continued operation when a network partition occurs.
- Therefore, CA is not achievable during an actual network partition in a distributed system.

Comparison of CAP Trade-offs

CAP combination	Behaviour during partition	Limitation
CP	Maintains consistent data	Some transactions may be rejected or delayed
AP	Continues accepting requests	Temporary inconsistency may occur
CA	Maintains consistency and availability	Cannot tolerate network partitions

Conclusion

The CAP theorem helps in understanding the limitations of distributed blockchain systems during network failures. For financial and blockchain applications, consistency is generally important because conflicting transactions can result in double spending and incorrect balances. Therefore, a CP-oriented approach is often preferred when transaction correctness is more important than continuous availability. The choice ultimately depends on the application's requirements.

Q23. Apply the concepts of cryptocurrencies and crypto assets to select a suitable blockchain-based payment solution for an international transaction scenario. Justify your selection. (10 Marks) - Module 5

Introduction

Cryptocurrency is a digital currency that uses cryptographic techniques and blockchain technology to enable secure peer-to-peer transactions without necessarily requiring traditional banking intermediaries.

Crypto assets are digital assets that exist on a blockchain and can represent currencies, tokens, ownership rights or other forms of value.

International Transaction Scenario

Consider a company in India that needs to transfer payment of $10,000 to a supplier in the United States for purchasing software services.

Using traditional banking systems may involve:

- High international transaction charges.
- Currency conversion fees.
- Delays due to intermediary banks.
- Limited availability outside banking hours.
- Additional documentation and processing.

A blockchain-based payment solution can reduce these limitations.

Suitable Solution: Stablecoin-Based International Payment

A suitable solution is to use a USD-backed stablecoin such as USDC (USD Coin) for the international payment.

USDC is a cryptocurrency designed to maintain a value close to one US dollar. Unlike highly volatile cryptocurrencies such as Bitcoin, its value is intended to remain stable relative to the US dollar.

Working of the Proposed Payment Solution

1. Currency Conversion: The Indian company converts the required amount into USDC through a compliant cryptocurrency service provider, subject to applicable regulations.
2. Wallet Creation: The company stores USDC in a digital wallet.
3. Transaction Initiation: The company enters the supplier's blockchain wallet address and initiates the payment.
4. Blockchain Validation: The transaction is broadcast to the blockchain network, where participating validators verify it.
5. Transaction Recording: Once validated, the payment is permanently recorded on the blockchain.
6. Payment Reception: The US supplier receives USDC in their digital wallet.
7. Conversion into Fiat Currency: The supplier can convert USDC into US dollars through a compliant exchange or payment provider.

Justification for Selecting USDC

1. Price Stability: USDC is designed to maintain a 1:1 value with the US dollar, reducing the price volatility associated with cryptocurrencies such as Bitcoin.

2. Faster Transactions: Blockchain-based transfers can be completed much faster than conventional international bank transfers, depending on the network and service provider.

3. Lower Transaction Costs: It can reduce intermediary banking charges and international transfer fees, although blockchain network and conversion fees still apply.

4. Transparency: Transactions are recorded on a public blockchain and can be independently verified.

5. Security: Cryptographic techniques and blockchain validation help protect transactions against unauthorized modifications.

6. Global Accessibility: Payments can be initiated and received across geographical boundaries without depending entirely on traditional banking hours.

Limitations

- Cryptocurrency regulations differ across countries.
- Wallet addresses must be entered correctly because transactions are generally irreversible.
- Blockchain network fees may fluctuate.

- Stablecoins involve issuer, reserve and redemption risks.
- Currency conversion and compliance requirements must be considered, particularly for Indian cross-border transactions.

Conclusion

USDC is a suitable blockchain-based payment option for international transactions because it combines the advantages of cryptocurrency with the relative price stability of a US dollar-backed digital asset. It provides transparency, faster settlement possibilities and reduced dependence on intermediary banking systems. However, regulatory compliance, conversion charges and the reliability of the service provider must be considered before implementation.

Q20. Demonstrate how Hyperledger Fabric can be applied to implement a blockchain-based document verification system involving multiple organizations. (10 Marks) -Module 4

Introduction

Hyperledger Fabric is an open-source, permissioned blockchain framework designed for enterprise applications. Unlike public blockchains, it allows only authorized participants to join the network.

It provides features such as:

- Permissioned membership.
- Smart contracts (Chaincode).
- Privacy and confidentiality.
- Distributed ledger.
- Consensus mechanisms.
- Identity management.

These features make Hyperledger Fabric suitable for implementing a document verification system involving multiple organizations.

Problem Statement

Consider a document verification system involving three organizations:

1. University: Issues academic certificates to students.
2. Employer: Requests and verifies academic certificates during recruitment.
3. Verification Agency: Checks the authenticity of submitted certificates.

In a traditional system, employers need to contact universities manually to verify certificates. This process is time-consuming and may involve forged documents.

Hyperledger Fabric can automate this process through a shared, tamper-evident ledger.

Proposed Architecture

The system consists of the following components:

1. Organizations

Each participating organization has its own identity and permissions.

- University: Issues and registers certificates.
- Employer: Requests certificate verification.
- Verification Agency: Performs independent verification.

2. Peers

Peers maintain copies of the distributed ledger and execute smart contracts. They participate in transaction validation according to the network configuration.

3. Ordering Service

The ordering service establishes the order of transactions and distributes blocks to participating peers.

4. Certificate Authority (CA)

The Certificate Authority manages digital identities and issues certificates to authorized participants.

5. Chaincode (Smart Contract)

Chaincode contains the business logic for issuing, registering and verifying documents.

6. Distributed Ledger

The ledger stores document-related records and transaction history, allowing authorized participants to verify information.

Working of the Document Verification System

Step 1: User Registration

The university, employer and verification agency register themselves with the Hyperledger Fabric network. Each organization receives a digital identity through the Certificate Authority.

Step 2: Document Issuance

The university issues a certificate to a student in PDF format.

Step 3: Hash Generation

The system generates a unique cryptographic hash (such as SHA-256) of the certificate.

For example:

- Original certificate → Hash: `A7F82C91...`
- Modified certificate → Hash: `B9D31E45...`

Even a small modification in the document produces a different hash.

Step 4: Recording on Blockchain

The university submits the certificate details and its hash to the blockchain through a smart contract.

The blockchain stores details such as:

- Certificate ID.
- Student ID.
- Issuing university.
- Issue date.
- Document hash.
- Verification status.

Step 5: Verification Request

When the student submits the certificate to an employer, the employer uploads the document to the verification application.

The application generates its hash and sends a verification request to the blockchain.

Step 6: Smart Contract Execution

The smart contract compares the uploaded document's hash with the hash already stored on the blockchain.

- If both hashes match, the document is considered unchanged since registration.
- If the hashes differ, the document is flagged as modified or inconsistent.
- If the certificate record does not exist, it is reported as unregistered.

Step 7: Verification Result

The employer receives the verification result without requiring lengthy manual communication with the university.

Sample Chaincode (Smart Contract)

The following simplified Go code demonstrates document registration and verification in Hyperledger Fabric.

```
package main

import (
    "encoding/json"
    "fmt"

    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)

type DocumentContract struct {
    contractapi.Contract
}

type Document struct {
    ID        string `json:"id"`
    StudentID string `json:"studentId"`
    Issuer    string `json:"issuer"`
    Hash      string `json:"hash"`
}

func (c *DocumentContract) RegisterDocument(
    ctx contractapi.TransactionContextInterface,
    id, studentID, issuer, hash string,
) error {

    exists, err := ctx.GetStub().GetState(id)
    if err != nil {
        return err
    }

    if exists != nil {
        return fmt.Errorf("document already exists")
    }

    doc := Document{
        ID: id,
        StudentID: studentID,
        Issuer: issuer,
        Hash: hash,
```

```
    }

    data, err := json.Marshal(doc)
    if err != nil {
        return err
    }

    return ctx.GetStub().PutState(id, data)
}

func (c *DocumentContract) VerifyDocument(
    ctx contractapi.TransactionContextInterface,
    id, submittedHash string,
) (bool, error) {

    data, err := ctx.GetStub().GetState(id)
    if err != nil {
        return false, err
    }

    if data == nil {
        return false, fmt.Errorf("document not found")
    }

    var doc Document
    err = json.Unmarshal(data, &doc)
    if err != nil {
        return false, err
    }

    return doc.Hash == submittedHash, nil
}
```

Note: This is illustrative chaincode logic. A production system must also enforce organization-level access control, issuer authorization, certificate revocation and appropriate endorsement policies.

Advantages of the Proposed System

1. Tamper Resistance: Once a document hash is recorded, unauthorized modifications can be detected.
2. Transparency: Authorized organizations can independently verify document records.
3. Faster Verification: Automated smart contracts reduce manual verification time.

4. Security: Digital identities and permission-based access restrict unauthorized participation.
5. Reduced Fraud: Fake or modified certificates can be identified through hash comparison.
6. Traceability: Every approved blockchain transaction has a verifiable transaction history.

Conclusion

Hyperledger Fabric provides an effective framework for building a secure, permissioned document verification system involving multiple organizations. By using digital identities, distributed ledgers, cryptographic hashing and smart contracts, universities can issue certificates and employers can independently verify their authenticity. This reduces manual effort, improves trust between organizations and helps prevent document forgery.